



University of Puerto Rico Mayagüez Campus
Electrical and Computer Engineering Department

Python: fast or slow?

Comparison in Performance of Pure Python vs. NumPy

Prepared by:

Jael A. Gonzalez De Arce
Néstor O. Santos Torres
Jayson D. Perez Ramirez
Francisco J Rivera Color
Adalberto J Roman Rivera

Date:

February 22, 2026

Abstract

This report compares the performance between using plain Python and the NumPy library for doing complex calculations, specifically multiplying vectors (1D-Arrays) and matrixes. When it comes to scripting languages, Python is incredibly popular as it offers the perfect balance between development speed and the ability to implement complex logical tasks with much ease.

However, when performing computationally intensive mathematical operations, such as large-scale vectors or matrix multiplications, pure Python can be quite slow. This is because Python's interpreted nature and object overhead make it inefficient for handling large numerical datasets. To overcome this limitation, libraries such as NumPy, which are implemented in C, are used.

For the methodology of the experiment, both vectors and matrices of different sizes were multiplied. The vector sizes ranged from 10 to 500 elements, while the matrices ranged from 10×10 to 500×500 . Using Python's timing functions, the execution time for each multiplication was measured.

After analyzing the results, while the speed difference is close to negligible arrays with small elements, when it comes to larger datasets, NumPy significantly outperforms plain Python. Python's execution time for multiplications, specifically in larger matrixes, increases dramatically while NumPy maintains a consistent low execution time. These results were expected, as NumPy's is implemented in mostly C (with some parts in C++), a low-level language that is more efficient and optimized, specifically as it is compiled and not interpreted.

Table of Contents

Table of Contents	2
I. Introduction	3
II. Theory	4
III. Discussion	4
III.1 Hypothesis.....	5
III.2 Design Choices / Methodology	5
III.3 Measurements	6
III.4 Results and Analysis	7
III.4.1 Vector Multiplication	7
III.4.2 Matrix Multiplication	7
IV. Conclusion	8
References	9
Appendix	10
A. Measured Execution Times	10
B. Plotted Execution Times	11
C. Plotted Speedup Ratios	12
D. Project Breakdown Structure	14
E. Use of Generative AI.....	14

I. Introduction

Python is a widely used programming language known for its readability and flexibility. However, its interpreted nature can make large-scale numerical computations slow. Libraries like NumPy address this by providing high-performance array operations through compiled code and vectorization.

The purpose of this study is to compare the performance of pure Python and NumPy in executing vector and matrix multiplications, quantifying how runtime scales with input size and the speedup between each. The scope of this project includes:

- Implementing element-wise multiplication of vectors (1D arrays) and standard matrix multiplication (2D arrays) in both pure Python and NumPy.
- Measuring and analyzing execution times for a range of vector lengths and matrix dimensions.
- Evaluating the performance differences and highlighting the advantages of using NumPy for computationally intensive tasks.

This analysis provides insight into when and why NumPy should be preferred over plain Python for numerical operations.

II. Theory

Python is an interpreted programming language known for its simplicity and ease of use. Because its syntax at times closely resembles spoken language, it is an “ideal language for scripting and rapid application development in many areas on most platforms.” [1] However, due to being interpreted and dynamically typed, it has additional computation cost than statically typed languages. Because of this, Python is often referred to as a slow language, especially when compared to low-level programming languages [2].

To overcome this bottleneck, developers typically use NumPy for complex mathematical operations. Described as “the fundamental package for scientific computing in Python” [3]. NumPy is primarily written in C, a low-level language that avoids the performance limitations of Python’s interpreted nature. Additionally, NumPy leverages vectorization to efficiently perform computations on entire arrays at once, eliminating the need for explicit loops in plain Python code [4].

III. Discussion

III.1 Hypothesis

The hypothesis of this experiment is that NumPy will outperform pure Python in numerical computations, particularly in operations involving large arrays and matrices. This is because NumPy internally uses compiled C, which executes mathematical operations at native machine speed and uses vectorization to perform calculations at entire set of values at the same time without the need for-looping for each value.

In contrast, pure Python executes arithmetic operations within the interpreter, adding substantial overhead per operation due to dynamic type checking and function call costs. Moreover, in plain python the computations for set of values are not done via vectorization. Therefore, while both implementations should yield identical results in correctness, NumPy is expected to achieve orders of magnitude faster execution times, especially as data size scales.

III.2 Design Choices / Methodology

To validate this hypothesis, two computational tasks were selected:

- **Vector (1-D array) element-wise multiplication.**
- **Matrix (2-D array) true multiplication**, using the standard triple-nested loop algorithm.

Both tasks were implemented twice: first using plain Python loops and then using NumPy's vectorized operations.

The chosen data sizes were designed to clearly illustrate performance scaling:

- **Vectors with lengths of 10, 50, 100, 200, and 500 elements.**
- **Matrices of size 10×10, 50×50, 100×100, 200×200, and 500×500.**

All random input arrays were generated with a fixed seed (`np.random.seed(42)`) to ensure reproducibility across all runs. The arrays were composed of 64-bit floating-point numbers (`float64`) to maintain consistency and precision across both implementations

A “warm-up” execution was performed before each timing to minimize cache and memory allocation overhead. Additionally, fifteen independent trials were conducted per data size to reduce variance from system load or background processes. The median runtime across trials was used as the representative execution time, as it is robust against occasional outliers.

Finally, correctness between Python and NumPy implementations was verified using NumPy's `np.allclose()` function, ensuring that both produced numerically equivalent results within floating-point tolerance.

III.3 Measurements

After implementing both the pure Python and NumPy versions of vector and matrix multiplication, the performance data was collected within the same script.

For each array or matrix size, the code:

1. Executed multiple timed trials using the `time.perf_counter()` function, which provides high-resolution wall-clock timing in seconds.
2. Stored all individual trial durations for both the Python and NumPy implementations in separate result lists.
3. Computed the median runtime across the fifteen trials for each data size to obtain a stable measurement representative of the typical case.
4. Recorded the medians in tuples of the form `(size, python_time, numpy_time)` for later comparison.

Once all sizes were processed, the collected data were printed to verify correctness and then used to produce the final tables. The tables created from the results can be seen in **Appendix A**. The table demonstrates the size of the data (vector size or matrix dimensions), followed by the time in plain Python and in NumPy in seconds.

Using these stored results, Matplotlib, a library for visualization of data in Python [5], was used to visualize the data, using line plots to compare performance trends. The graphs generated can be seen in **Appendix B**. Separate figures were generated for:

1. **Vector multiplication:** Median runtime (Python vs. NumPy) vs. vector size
2. **Matrix multiplication:** Median runtime (Python vs. NumPy) vs. matrix dimension (logarithmic y-scale)

Finally, after obtaining all timing data, graphs were generated to illustrate the speedup ratio for both vector and matrix multiplications as a function of their dimensions (**Appendix C**). The speedup was calculated using the following relationship:

$$Speedup = \frac{T_{Python}}{T_{NumPy}}$$

These plots clearly show the scaling relationship between data size and runtime, allowing direct visual comparison of efficiency between both implementations.

All measurements were collected on the same system under consistent conditions, ensuring fair comparison. No input/output operations, data creation, or plotting were included in the timed regions.

III.4 Results and Analysis

The collected data clearly demonstrates a substantial performance difference between pure Python and NumPy for both vector and matrix multiplication.

III.4.1 Vector Multiplication

From **Table A.1**, the execution times for vector element-wise multiplication show that for very small vectors (e.g., size 10), Python and NumPy times are relatively close, with Python slightly slower. However, as the vector size increases, Python's execution time grows significantly, while NumPy remains almost constant.

- **Small Vectors ($n \leq 50$):** The difference in execution time is minimal. Python is slightly slower than NumPy, but the gap is not dramatic.
- **Medium to Large Vectors ($n \geq 100$):** The execution time of Python begins to increase more rapidly, whereas NumPy maintains near-constant performance.

From **Figure C.1**, which plots the speedup ratio of NumPy over Python, we observe that the speedup grows with vector size. For example, at $n = 10$, the speedup is small, ~ 1.6 , but for $n = 500$, the speedup reaches a substantial factor of ~ 48.6 . This trend clearly illustrates the advantage of NumPy's vectorization: performing operations on the entire array at once drastically reduces overhead, even when the computations take only in one-dimensional arrays.

III.4.2 Matrix Multiplication

Matrix multiplication, being a more computationally intensive operation, shows an even more pronounced difference. **Table A.2** reveals that for a 10×10 matrix, Python is already slower than NumPy by a noticeable margin, and this difference grows exponentially with matrix size.

- **Small Matrices (10×10):** Python takes ~ 171.3 times longer than NumPy. While the absolute time is small, the relative performance difference is noticeable.
- **Medium Matrices (50×50 to 100×100):** The Python execution time increases steeply (seconds), whereas NumPy execution remains in microseconds to milliseconds range.
- **Large Matrices (200×200 to 500×500):** The performance gap becomes dramatic. For the 500×500 matrix, Python takes over *20 seconds*, while NumPy completes the same operation in just $\sim 1.14 \times 10^{-3}$ seconds.

Figure C.2 illustrates the speedup ratio for matrices. NumPy's advantage scales sharply with matrix dimensions. Small matrices show a speedup of ~ 171.3 , a higher value than that of the largest speedup ratio in the one-dimensional arrays. Larger matrices exhibit massive speedups exceeding $\sim 17,766$. This is a direct consequence of NumPy's low-level optimizations and avoidance of Python's interpreter overhead in nested loops.

IV. Conclusion

This experiment confirms that NumPy significantly outperforms pure Python in numerical computations. While Python is simple and expressive, its interpreted nature introduces overhead that makes it inefficient for large-scale mathematical operations. NumPy, on the other hand, executes most computations in compiled code with vectorization, resulting in far greater speed.

For small datasets, both approaches perform similarly, but as data size grows, NumPy's efficiency becomes evident. For instance, in this study, the multiplication of a 500×500 matrix showed that NumPy was approximately *17,766.9 times* faster than plain Python. In real-world scenarios, datasets often range from thousands to millions of rows or columns in scientific computing, machine learning, or big data applications, where such performance differences are noticeable and critical. This underscores a key engineering principle: achieving high performance in Python depends on leveraging specialized numerical libraries like NumPy rather than relying solely on pure interpreter-based code.

While these results demonstrate that Python is *slower* in these scenarios, it is important to note that this observation applies specifically to computationally intensive numeric operations and does not reflect Python's performance in general-purpose programming, as this is beyond the scope of the experiment.

References

- [1] Python Software Foundation, “Python Language Reference,” [Online]. Available: <https://www.python.org>. [Accessed: Feb. 23, 2026].
- [2] O. Dekel, “Optimizing Python: Why Python is Slow & 4 Optimization Methods,” Intel Granulate, [Online]. Available: <https://granulate.io/blog/optimizing-python-why-python-is-slow-optimization-methods/>. [Accessed: Feb. 23, 2026].
- [3] NumPy Developers, “What is NumPy? – NumPy v2.2 Manual,” [Online]. Available: <https://numpy.org/doc/2.2/user/whatismumpy.html>. [Accessed: Feb. 23, 2026].
- [4] Practical Data Science, “Vectorization in Python Notebooks,” [Online]. Available: https://www.practicaldatascience.org/notebooks/class_2/week_4/11_vectorization.html. [Accessed: Feb. 23, 2026].
- [5] Matplotlib Developers, “Matplotlib: Visualization with Python,” [Online]. Available: <https://matplotlib.org/>. [Accessed: Feb. 23, 2026].
- [6] OpenAI, “ChatGPT (GPT-5),” [Online]. Available: <https://chat.openai.com>. [Accessed: Feb. 23, 2026].

Appendix

A. Measured Execution Times

Vector Size (n)	Python Time (s)	NumPy Time (s)
10	7.96×10^{-7}	4.95×10^{-7}
50	2.88×10^{-6}	5.09×10^{-7}
100	5.43×10^{-6}	4.93×10^{-7}
200	1.07×10^{-5}	5.50×10^{-7}
500	2.81×10^{-5}	5.78×10^{-7}

Table A.1: Execution times for Vector multiplication

Matrix Size (n×n)	Python Time (s)	NumPy Time (s)
10×10	1.78×10^{-4}	1.04×10^{-6}
50×50	2.09×10^{-2}	5.08×10^{-6}
100×100	1.63×10^{-1}	3.18×10^{-5}
200×200	1.35	8.35×10^{-5}
500×500	20.22	1.14×10^{-3}

Table A.2: Execution times for Matrix multiplication

B. Plotted Execution Times

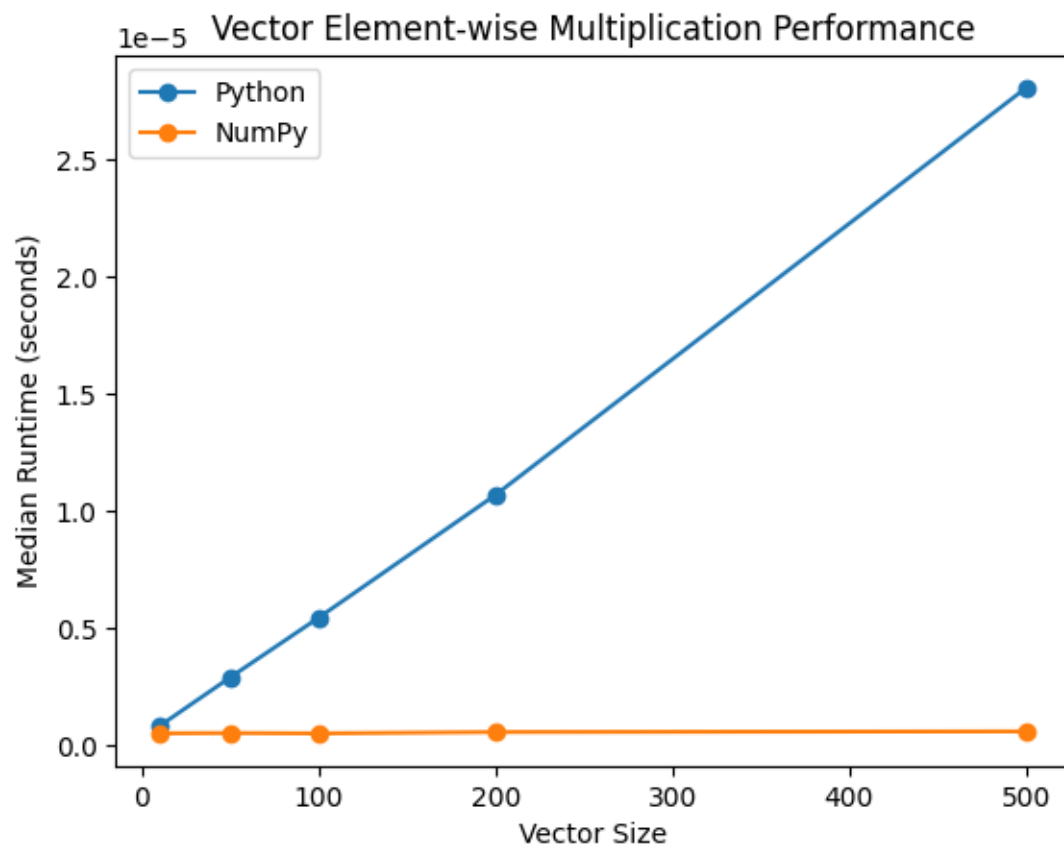


Figure B.1: Vector Element-wise Multiplication Performance

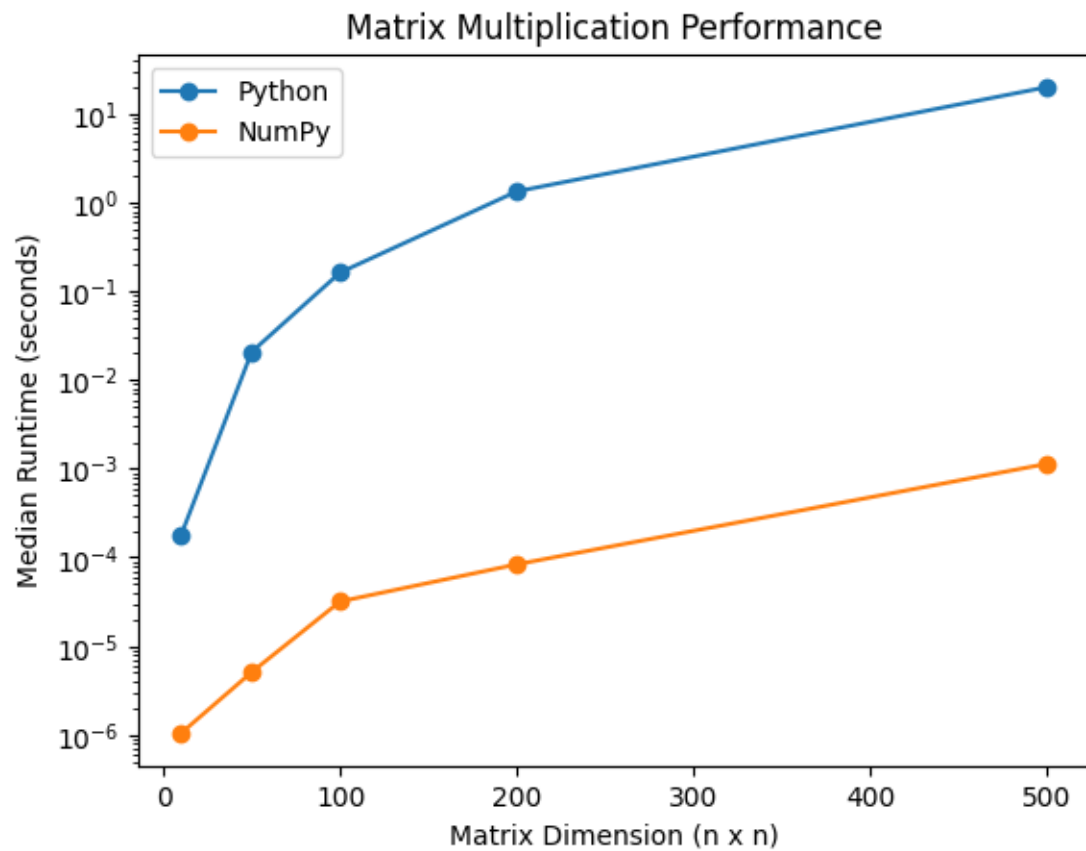


Figure B.2: Matrix Multiplication Performance

C. Plotted Speedup Ratios

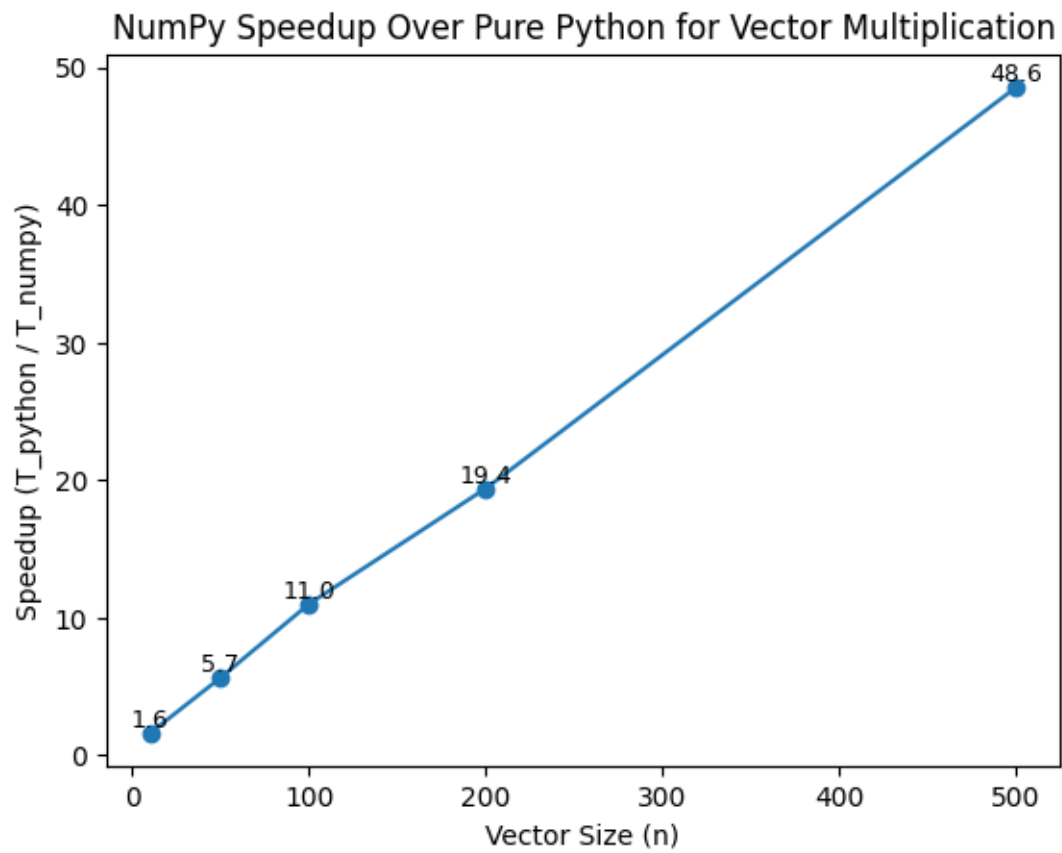


Figure C.1: NumPy Speedup Over Pure Python for Vector Multiplication

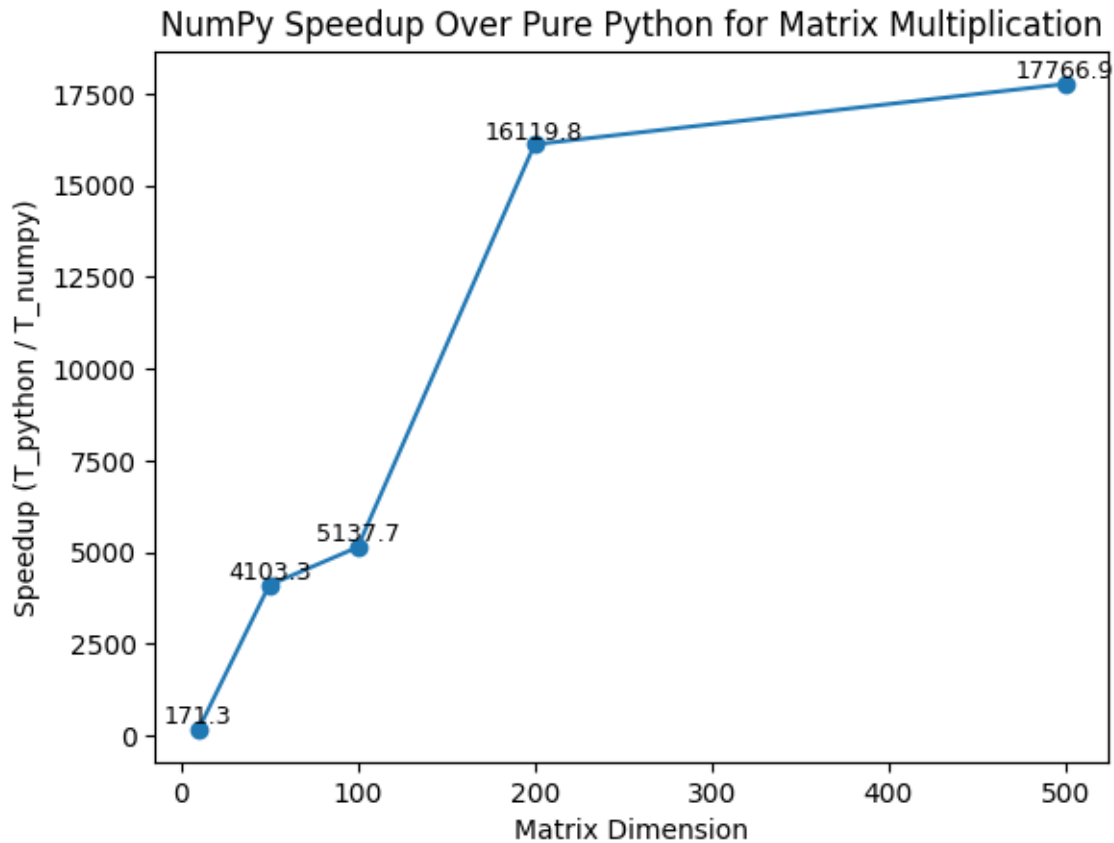


Figure C.2: NumPy Speedup Over Pure Python for Matrix Multiplication

D. Project Breakdown Structure

The completion of this project was a collaborative effort by all team members. Each member contributed to all stages of the work, including the design, implementation, testing, and validation of both the Python and NumPy code. Additionally, all members participated in compiling, reviewing, and refining the written report, as well as preparing the accompanying presentation and project video.

E. Use of Generative AI

Generative AI tools were employed during the preparation of this project, but only as a supplementary aid to revise and rewrite certain sections for improved clarity, readability, and presentation. No part of the report, code, or supporting materials was solely generated by AI. All content that involved AI assistance was carefully reviewed, verified, and validated by the team.